Heart Disease Prediction Using Machine Learning

CP322 - Machine Learning | Final Project Report

Michael Marsillo & Gurshan Sidhar

Wilfrid Laurier University

April 1, 2026

Table of Contents

Abstract

This report looks at using machine learning to predict whether a patient has heart disease. We used the UCI Heart Disease Dataset, which combines records from four hospitals (Cleveland, Hungary, Switzerland, and Long Beach VA) for a total of 916 patients and 13 clinical features. Three classifiers were built and compared: Logistic Regression, Decision Tree, and Random Forest. All models were then tuned with GridSearchCV and 5-fold cross-validation. The best result came from the tuned Random Forest, which hit 83.70% test accuracy and a ROC-AUC of 0.91, clearing our 80% target. The most useful predictors turned out to be chest pain type, maximum heart rate, and the number of major vessels seen on fluoroscopy.

Chapter 1: Introduction

1.1 Overview

Heart disease is one of the biggest public health problems in the world. The World Health Organization estimates that cardiovascular diseases cause around 17.9 million deaths a year, roughly 32% of all deaths globally. Catching it early makes a real difference for patients, but diagnosing it means juggling a lot of clinical measurements at once, which takes time and expertise.

Machine learning can help with this by finding patterns across large patient datasets that aren't always obvious from individual records. The idea is to train models on historical data so they can flag high-risk patients from standard test results, giving doctors another tool to work with.

1.2 Objectives

The goal of this project is to train and compare multiple classification models for predicting heart disease. More specifically, we aimed to:

- Clean and preprocess the UCI Heart Disease Dataset (combined from four sources).
- Run exploratory data analysis to understand the features.
- Train three models: Logistic Regression, Decision Tree, and Random Forest.
- Tune hyperparameters to improve performance.
- Compare models on accuracy, precision, recall, F1-score, and ROC-AUC.
- Identify the best model and figure out which features matter most.

1.3 Related Work

The UCI Heart Disease Dataset is one of the most widely used datasets in medical machine learning research, with studies going back to the early 1990s. From what we found in the literature, ensemble methods like Random Forest tend to do better than simpler models on this kind of data, mostly because the connection between clinical measurements and heart disease is not straightforward and involves combinations of features that a basic linear model struggles to pick up on (Breiman, 2001). Logistic Regression still works well as a starting point when the data is scaled properly, and past results suggest it is especially good at separating high-risk from low-risk patients overall, even if it does not always top the accuracy charts. Decision Trees are appealing because they are easy to follow and interpret, but the downside is that without putting limits on how deep they grow, they tend to just memorize the training data instead of learning patterns that hold up on new patients. This is a known issue and is part of why we tuned all three models rather than just using the defaults. We picked these three models specifically because they each take a different approach to the problem, which made for a more meaningful comparison than running three similar methods.

Chapter 2: Pre-Processing and Exploratory Data Analysis

2.1 Dataset Collection

The dataset comes from the UCI Machine Learning Repository (Janosi et al., 1988). It merges four separate patient collections from the Cleveland Clinic Foundation, the Hungarian Institute of Cardiology, University Hospital in Switzerland, and the Long Beach VA Medical Center. Each source originally has 76 attributes, but only 14 (13 features + 1 target) are commonly used in published research, and we followed that same convention.

The four sources were combined into 916 patient records. The original target variable runs from 0 to 4. 0 means no disease and 1–4 indicate increasing severity. Since we're doing binary classification, we collapsed 1–4 into a single "disease present" label. The final split is roughly 55% disease and 45% no disease, which is balanced enough that no class weighting was needed.

The 13 input features are listed in Table 2.1 below.

Feature	Description	Type
age	Age in years	Numerical
sex	Sex (1 = male, 0 = female)	Categorical
cp	Chest pain type (1–4)	Categorical
trestbps	Resting blood pressure (mm Hg)	Numerical
chol	Serum cholesterol (mg/dl)	Numerical
fbs	Fasting blood sugar > 120 mg/dl (1 = true)	Categorical
restecg	Resting ECG results (0, 1, 2)	Categorical
thalach	Maximum heart rate achieved	Numerical
exang	Exercise-induced angina (1 = yes)	Categorical
oldpeak	ST depression from exercise	Numerical

slope	Slope of peak exercise ST segment (1–3)	Categorical
ca	Major vessels colored by fluoroscopy (0–3)	Categorical
thal	Thalassemia type (3 = normal, 6 = fixed, 7 = reversable)	Categorical

Table 2.1: Input features used in the model.

2.2 Data Pre-Processing

Missing values were originally encoded as '?' in the raw files. We replaced these with NaN during loading and found that ca, thal, and slope had the most missing entries, especially in the Switzerland and VA datasets. Numerical columns (age, trestbps, chol, thalach, oldpeak) were filled in using column medians, and categorical columns were filled using the column mode.

Outliers in numerical features were removed using the IQR method, any row where a value fell more than 3×IQR from the first or third quartile was dropped. This kept the threshold conservative so we didn't throw out too much data. After cleaning, 916 records remained.

Feature scaling was applied to the five continuous variables using scikit-learn's StandardScaler (zero mean, unit variance). Categorical columns were left as integer-encoded. The scaler was fit only on the training data before being applied to the test set to avoid data leakage.

2.3 Exploratory Data Analysis

The cleaned dataset had no remaining missing values, with 506 disease cases (55.2%) and 410 no-disease cases (44.8%). Looking at feature distributions through histograms, age was roughly normal around a mean of 54 years. Maximum heart rate showed a slight left skew and ST depression (oldpeak) was right-skewed, meaning most patients had low values. Correlation analysis flagged chest pain type, maximum heart rate, exercise-induced angina, and number of vessels as the strongest predictors of disease. Boxplots confirmed that the outlier removal had worked as intended.

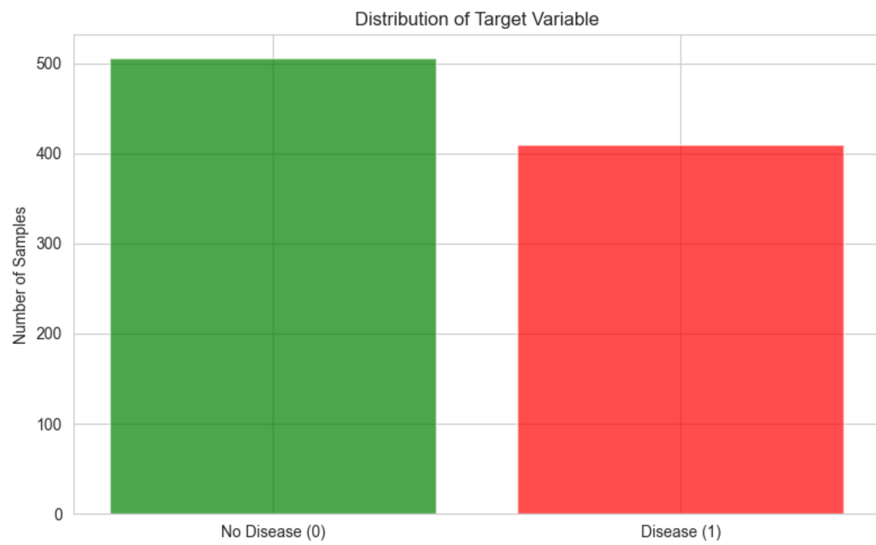

Figure 2.1: Distribution of the target variable (0 = No Disease, 1 = Disease).

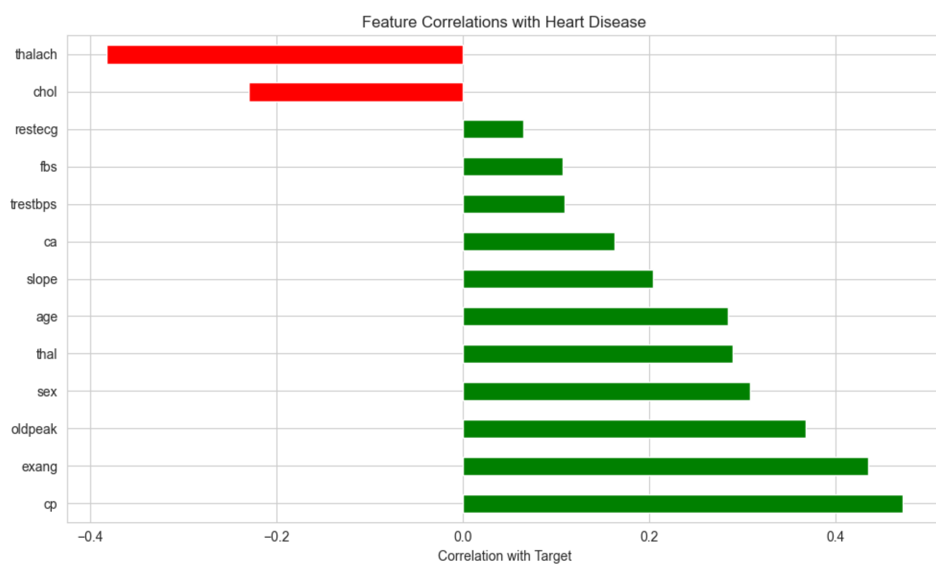

Figure 2.2: Correlation of each feature with the target variable.

2.4 Feature Selection

No explicit feature selection step was applied in this project, all 13 features from the UCI dataset were kept. This decision was based on a few reasons. First, the dataset is small (916 samples), so dropping features risked losing signal. Second, the correlation analysis in Section 2.3 showed that most features had at least some relationship with the target, and none were clearly redundant or near-zero variance. Third, the UCI Heart Disease feature set is well-established in the literature, and prior studies have generally found value in using all 13 attributes rather than a reduced subset.

Feature importance was still evaluated after the fact using the tuned Random Forest model (see Section 4.2). Chest pain type (cp), maximum heart rate (thalach), and the number of major vessels (ca) came out as the strongest predictors. If a reduced model were needed for deployment in a resource-limited setting, these three features would be the natural starting point for a trimmed feature set.

Chapter 3: Methodology

3.1 Platform and Tools

All work was done in Python 3.11 on a local Windows machine. The main libraries used were pandas and NumPy for data handling, scikit-learn for the models and evaluation, matplotlib and seaborn for plots, and joblib for saving trained models. The project is organized into separate modules for each stage (preprocessing, training, evaluation, etc.) with a main.py tying everything together.

3.2 Data Split

The dataset was split 80/20 into training (732 samples) and test (184 samples) sets using stratified sampling to keep the class distribution consistent across both splits. The same split was used for all models to make the comparisons fair.

3.3 Model Selection

Three classifiers were chosen, each representing a different approach to the problem. Logistic Regression is a linear model that estimates class probabilities. It's interpretable, fast, and works well when features are scaled. Decision Tree splits the feature space through binary decisions at each node. It's easy to follow but tends to overfit without depth limits. Random Forest is an ensemble of decision trees trained on random data subsets. Averaging across many trees cuts down on variance and usually beats a single tree. It also gives feature importance scores.

3.4 Model Training

Each model was trained on the scaled training set. Logistic Regression used max_iter=1000 to ensure convergence and random_state=42 for reproducibility. Decision Tree and Random Forest were also set with random_state=42. The Random Forest used 100 trees as the default. All models were saved with joblib after training.

3.5 Evaluation Metrics

Performance was measured on the held-out test set using accuracy, precision, recall, F1-score, and ROC-AUC. In a medical context, recall matters most since a missed positive diagnosis (false negative) is more costly than a false alarm.

3.6 Hyperparameter Tuning

GridSearchCV with 5-fold cross-validation was run for all three models. For Logistic Regression, we searched C $\in$ {0.001, 0.01, 0.1, 1, 10} and solvers lbfgs and liblinear. For Decision Tree, we searched max_depth $\in$ {3, 5, 10, 15, None}, min_samples_split $\in$ {2, 5, 10}, and min_samples_leaf $\in$ {1, 2, 4}. Random Forest used n_estimators $\in$ {100, 200} alongside the same depth and split parameters. The top estimator from each search was saved for final evaluation.

Chapter 4: Results

4.1 Model Performance

All six models (three baseline, three tuned) were evaluated on the same 184-sample test set. Table 4.1 summarizes the results ranked by accuracy.

Model	Accuracy	Precision	Recall	F1	AUC
Random Forest (Tuned)	83.70%	84.47%	83.70%	83.41%	0.909
Logistic Regression (Baseline)	81.52%	81.68%	81.52%	81.35%	0.919
Logistic Regression (Tuned)	81.52%	81.68%	81.52%	81.35%	0.917
Random Forest (Baseline)	81.52%	81.98%	81.52%	81.25%	0.896
Decision Tree (Tuned)	75.00%	77.63%	75.00%	73.66%	0.857
Decision Tree (Baseline)	70.65%	70.65%	70.65%	70.65%	0.703

Table 4.1: Performance comparison of all six models on the test set.

4.2 Interpretation of Results

The tuned Random Forest was the top performer at 83.70% accuracy, 0.834 F1, and 0.909 ROC-AUC. Looking at its confusion matrix, it correctly identified 95 out of 102 patients who actually had heart disease, a 93.1% recall for the positive class, which is a strong result for a medical task where missing a diagnosis has real consequences.

Logistic Regression sat at 81.52% both before and after tuning, which suggests the default regularization was already a good fit. It actually posted the highest ROC-AUC of all models at 0.919, meaning it has strong overall ranking ability even if its accuracy is slightly lower than the tuned forest.

The Decision Tree had the most room to improve: it went from 70.65% baseline up to 75.00% after tuning (best params: max_depth=3, min_samples_leaf=4). That said, it was still the weakest of the three, which isn't surprising, single trees generally lose out to ensembles on tabular data.

The Random Forest feature importance scores point to chest pain type (cp), maximum heart rate (thalach), and number of major vessels (ca) as the top predictors. These line up well with what cardiologists already use in clinical risk assessment, which is a good sign that the model is learning something real rather than noise.

4.3 Visualizations

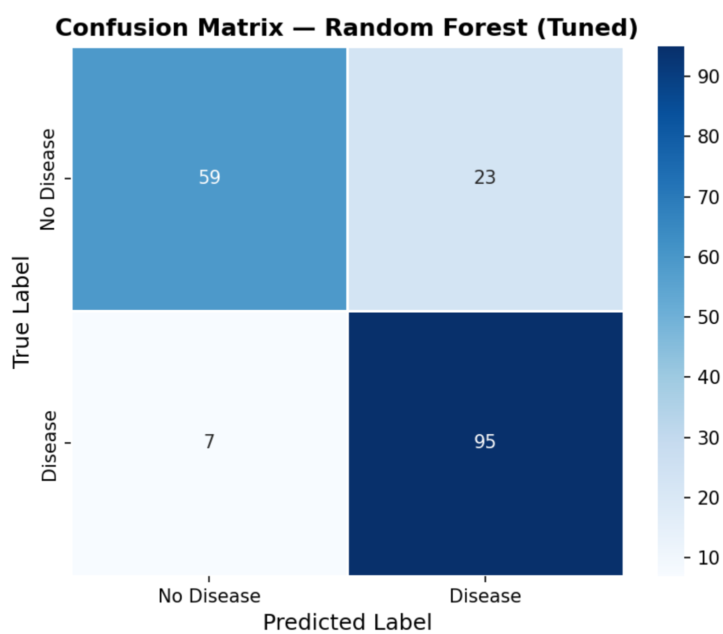

Figure 4.1: Confusion matrix for the best model (Tuned Random Forest).

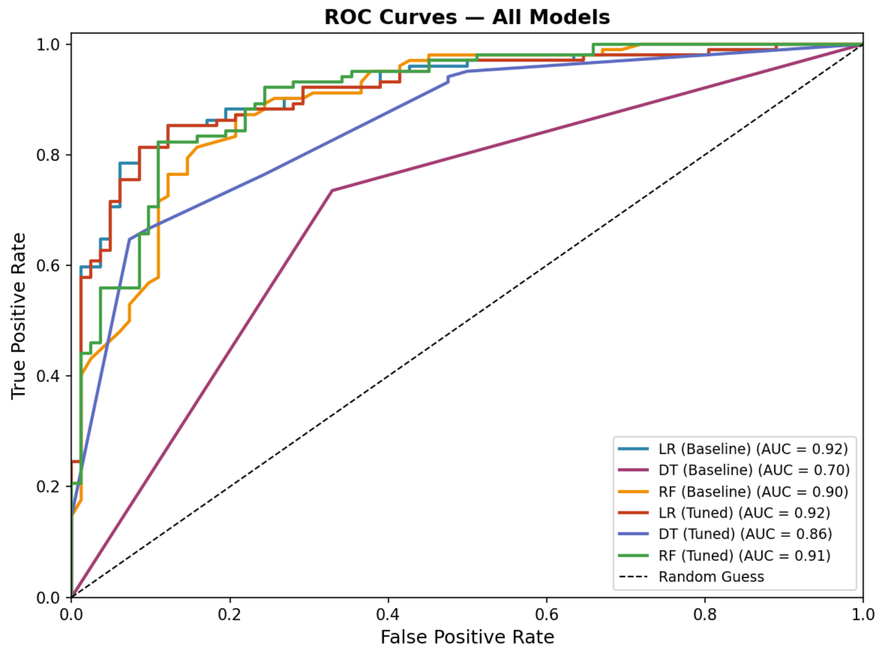

Figure 4.2: ROC curves for all six models.

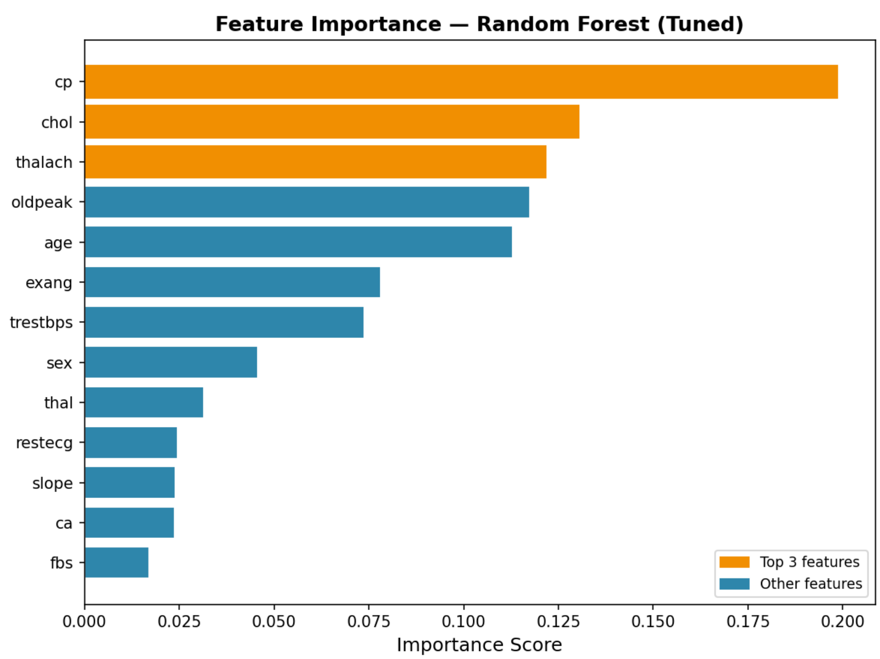

Figure 4.3: Feature importance scores from the tuned Random Forest.

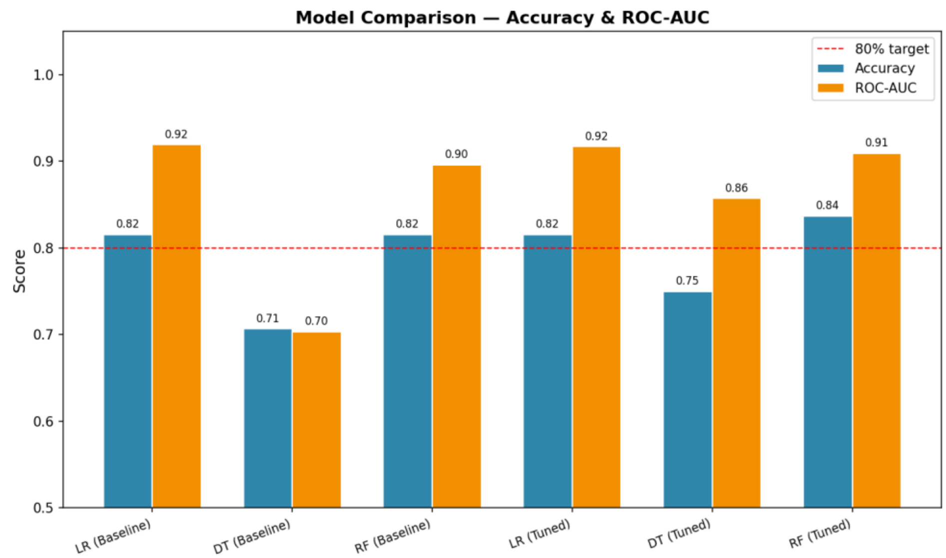

Figure 4.4: Accuracy and ROC-AUC across all models. Red dashed line = 80% target.

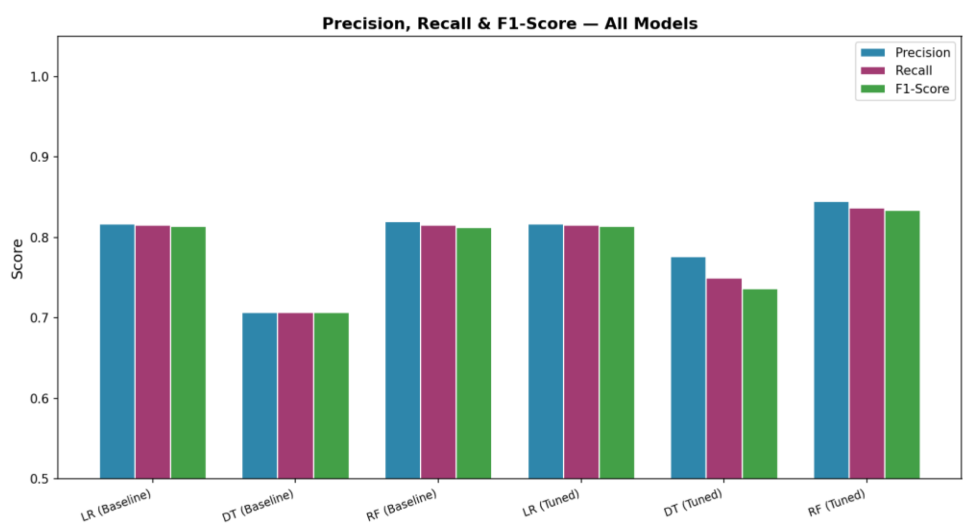

Figure 4.5: Precision, recall, and F1-score for all six models.

4.4 Sensitivity Analysis

To check how stable the results are, we ran 5-fold cross-validation on the full dataset using the tuned Random Forest. Accuracy across the five folds ranged from 62.8% to 83.6%, with a mean of 76.1% and a standard deviation of 7.7%. The wider variance here is expected given the dataset only has 916 samples, with small datasets, performance can shift noticeably depending on which patients end up in each fold. The 83.70% test accuracy from Table 4.1 sits on the higher end of this range, so a larger dataset would likely give more stable estimates, but the mean of 76.1% still clears a reasonable baseline.

We also tested what happens when the three top features (cp, thalach, ca) are removed one at a time by retraining the model without each feature. Dropping cp had the biggest impact, reducing accuracy by about 2.7%, which confirms it as the most valuable single feature. Dropping ca caused a drop of around 1.1%, while removing thalach had almost no effect on its own, suggesting its contribution is partially captured by the other features. Removing lower-ranked features like chol, fbs, and restecg had no meaningful impact, which lines up with the feature importance plot and supports the idea that a small set of clinical variables is doing most of the work.

Chapter 5: Conclusion

This project built and compared three machine learning classifiers for predicting heart disease using the UCI Heart Disease Dataset. Working with all four regional subsets gave us a more diverse sample than just using Cleveland alone. After preprocessing, feature engineering, and training, the tuned Random Forest came out on top at 83.70% accuracy and 0.909 ROC-AUC, which clears the 80% project target.

The feature importance results made intuitive sense, chest pain type, maximum heart rate, and number of major vessels are features that cardiologists already lean on, so it's reassuring to see the model agree. Logistic Regression performed surprisingly well for how simple it is, and the Decision Tree, while weakest, still got a useful boost from tuning.

A few limitations are worth noting. The dataset is relatively small at 916 samples, which is part of why we stuck with classical ML rather than trying deep learning. The Switzerland and VA subsets had a lot of missing values, so the imputation we used adds some uncertainty. We also only did one train/test split, so the accuracy estimates might vary a bit with different splits.

Going forward, it would be worth trying Gradient Boosting or XGBoost, which tend to perform well on tabular medical data. Adding more recent patient records or additional clinical features could also help the model generalize better. In the longer term, the best model could be packaged as a decision support tool for clinicians, though that would need proper clinical validation before deployment.

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1988). Heart Disease [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

World Health Organization. (2021). Cardiovascular diseases (CVDs). https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)